

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

ADAPTER PATTERN REPORT

CS202

Group members:

1. 25125030 - Võ Gia Minh
2. 25125055 - Phạm Huy Khánh
3. 25125066 - Nguyễn Quốc Thịnh
4. 25125085 - Chu Nguyễn Gia Khánh

21st August 2026



Contents

1	A Real-World Problem	2
2	A Naive Solution	2
3	Problems with the Naive Solution	2
4	Structure	3
4.1	Object Adapter Structure (Using Composition)	3
4.2	Class Adapter Structure (Using Inheritance)	4
5	Solution & Diagram Explanation	6
5.1	Solution Overview	6
5.2	Diagram Explanation	6
5.3	Data Flow	7
6	Implementation	7
7	Quizzes	11
8	References	14
9	Appendices	14

1 A Real-World Problem

Imagine that we are creating a stock market tracking application. The app currently uses a custom hook to download stock data from multiple sources in XML format, which it then provides to our UI components to display beautiful charts and diagrams to the user. At some point, we decide to improve the app by integrating a smart 3rd-party analytics library. But there is a catch: the analytics library's hook only works with data in JSON format.

We could change our existing components and data-fetching logic to work exclusively with JSON. However, this could break a lot of existing code that already relies on the XML structure, making this approach unfeasible.

We can solve this problem by creating an Adapter Hook to convert the data from XML format to JSON. We then adjust our UI components so they only communicate with the analytics library through this Adapter Hook. When a component uses this hook, the hook internally receives the XML data, translates it into JSON, passes the request to the appropriate methods of the 3rd-party analytics library, and returns the final data back to the component. This allows the new library to work flawlessly without breaking our existing app structure.

2 A Naive Solution

Instead of adapting the new tool to fit the system, the naive approach adapts the entire system to fit the new tool. This involves:

- Going into the core data-fetching hook and changing it to parse and return JSON instead of XML.
- Hunting down every single UI component, chart, and diagram that currently expects an XML structure and rewriting their internal logic to handle JSON.

3 Problems with the Naive Solution

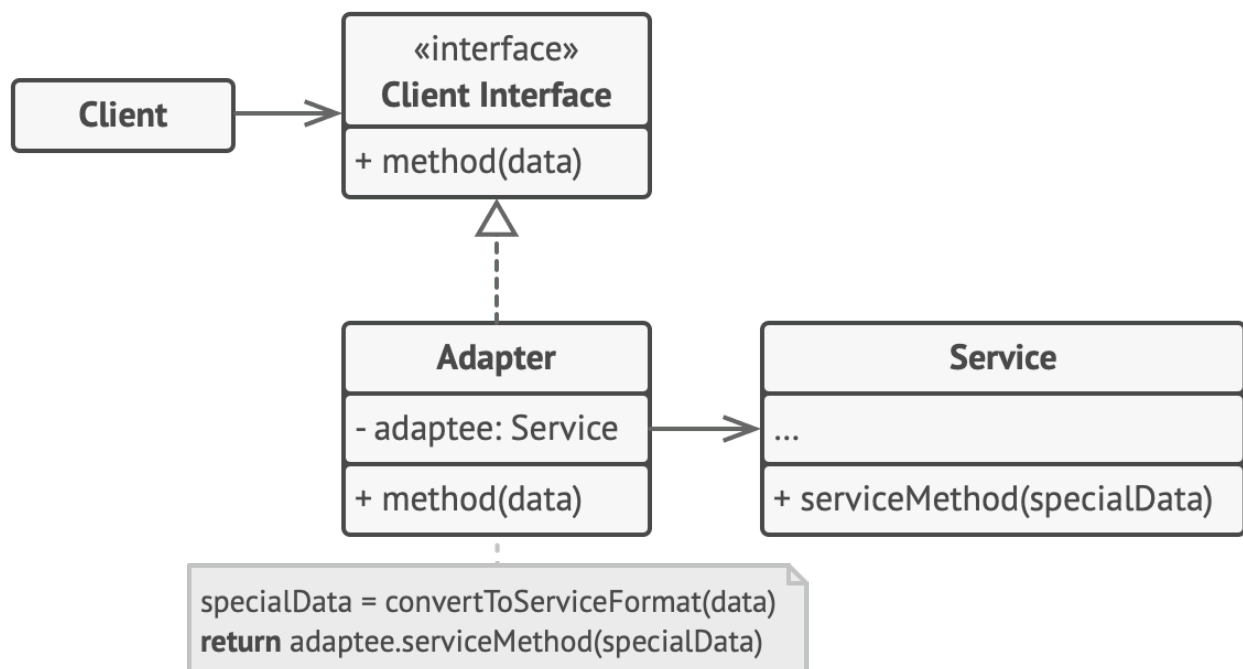
In software engineering, this approach violates several best practices (like the Open-Closed Principle) and introduces massive risks:

- **High Risk of Breaking Changes:** You are modifying legacy code that already works perfectly. Changing the fundamental data structure from XML to JSON across the entire app is practically guaranteed to introduce unexpected bugs.
- **Massive Refactoring Time:** Developers would have to spend hours or days rewriting and re-testing UI components just to maintain the exact same functionality the app had yesterday, rather than focusing on the new analytics features.
- **Tight Coupling to 3rd-Party Tools:** If you rewrite your core app to strictly use JSON just because this specific analytics library demands it, what happens if you switch to a different library in two years that requires a completely different format? You would have to rewrite the app all over again.

4 Structure

4.1 Object Adapter Structure (Using Composition)

This diagram illustrates the **Object Adapter** pattern, which relies on object composition to achieve compatibility. The adapter implements the interface of one object and wraps the other.

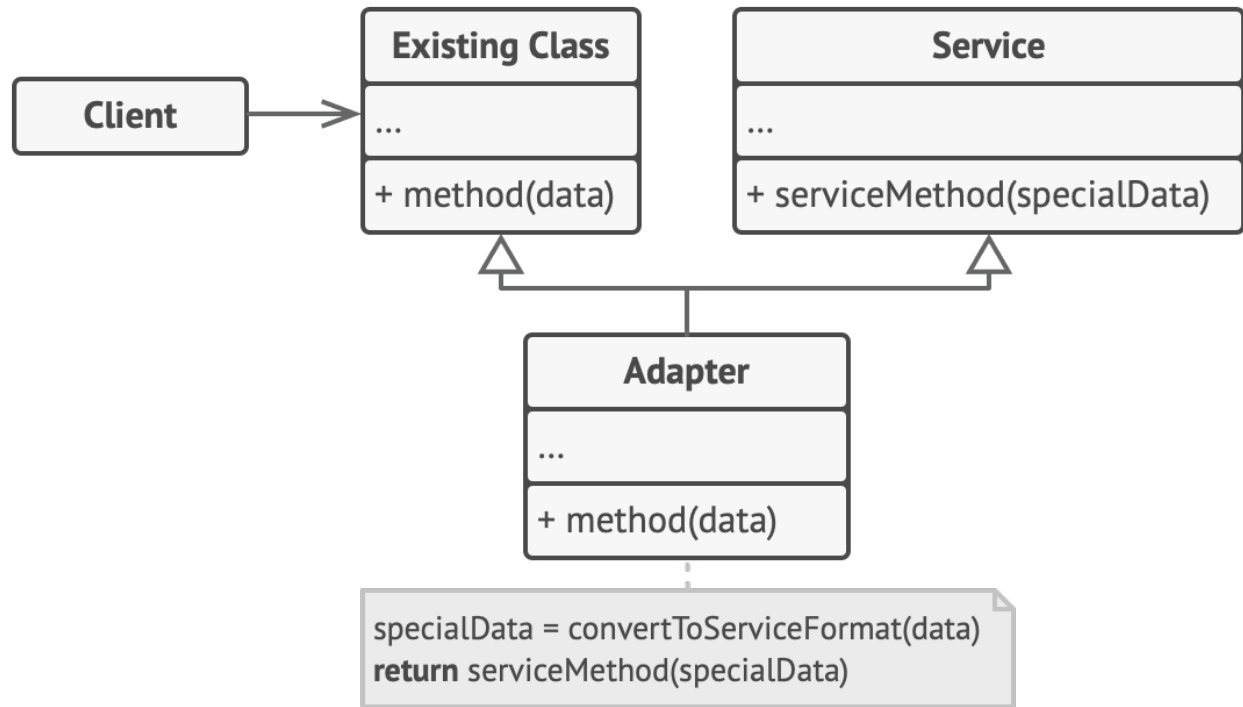


- **Client:** The existing business logic or application code. It interacts with other classes exclusively through the target **Client Interface**.
- **Client Interface:** An abstract definition, such as an interface or abstract class, that outlines the method signatures the Client expects, such as `method(data)`.
- **Service (The Adaptee):** An existing, useful class, such as a third-party library or legacy code, with an incompatible interface. The Client cannot use it directly because it expects `serviceMethod(specialData)`.
- **Adapter:** The central bridge class.
- **Implementation:** The Adapter implements the **Client Interface**, allowing the Client to interact with it seamlessly.
- **Composition:** The Adapter maintains a private reference, or wrapped object, to the **Service**, denoted as - `adaptee: Service`.
- **Execution Flow:** When the Client invokes `method(data)`, the Adapter intercepts the call. As shown in the attached note, the Adapter translates the incoming `data` into the required `specialData` format, and then delegates the execution to the wrapped object by calling `adaptee.serviceMethod(specialData)`.

4.2 Class Adapter Structure (Using Inheritance)

This diagram illustrates the **Class Adapter** pattern, which relies on multiple inheritance to achieve compatibility. Instead of wrapping an object, the adapter inherits behaviors from both the expected interface and the incompatible service.

Note: This approach requires a programming language that supports multiple inheritance, such as C++.

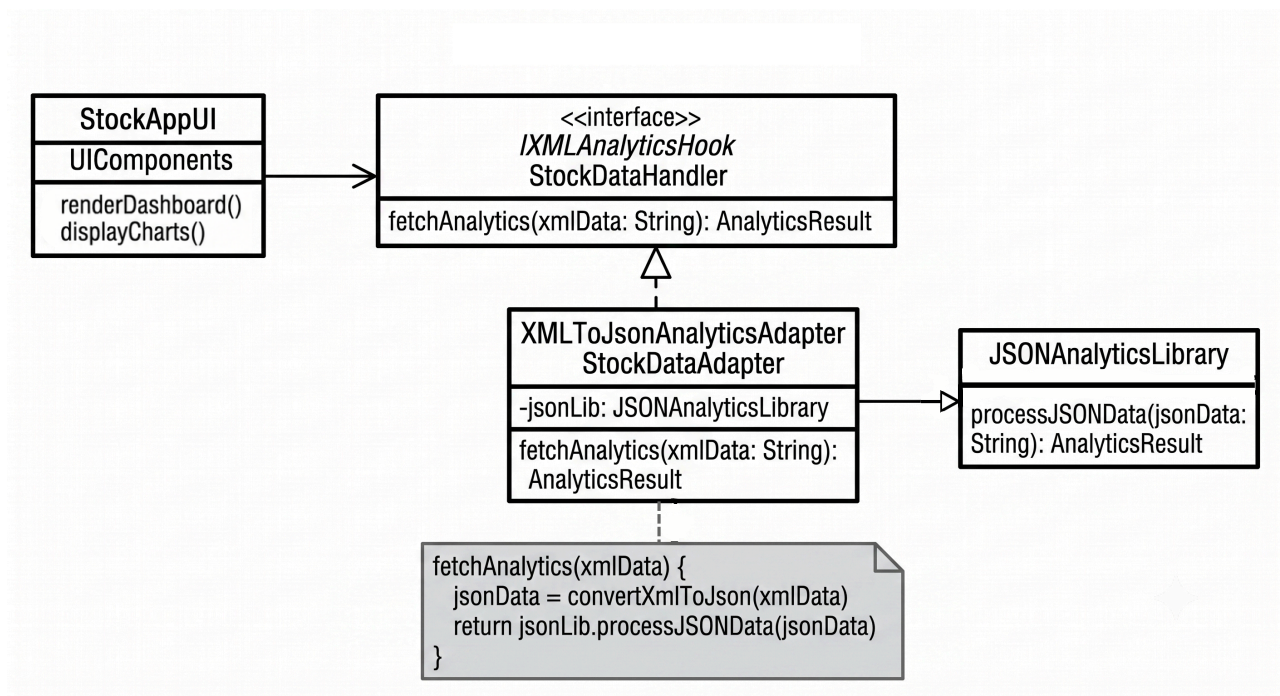


- **Client:** The existing application code that expects to interact with the system via the **Existing Class** structure.
- **Existing Class (Target):** The class or interface that defines the standard domain-specific behavior the Client expects, such as `method(data)`.
- **Service (The Adaptee):** The incompatible class containing the desired functionality, such as `serviceMethod(specialData)`.
- **Adapter:** The bridge class in this inheritance-based model.
- **Multiple Inheritance:** The Adapter simultaneously inherits from both the **Existing Class** and the **Service**. It does not wrap a separate Service object.
- **Execution Flow:** Because the Adapter inherits directly from the Service, it has native access to the Service's methods. When the Client calls `method(data)`, the Adapter translates the data and invokes the inherited method directly, as shown in the note: `return serviceMethod(specialData)`. There is no delegation to a separate **adaptee** object.

5 Solution & Diagram Explanation

5.1 Solution Overview

To solve the problem that was mentioned in the introduction without breaking the existing XML-based application structure, we implement the **Adapter Design Pattern**. Instead of rewriting our existing data-fetching logic and UI components to handle JSON exclusively, we use an intermediary component, the Adapter, that acts as a translator between the existing XML system and the new JSON library.



5.2 Diagram Explanation

Based on the provided UML diagram, the solution is structured with the following key components:

- **Client (StockAppUI)**: Represents the application's existing user interface components. This class contains methods like `renderDashboard()` and `displayCharts()`. `StockAppUI` continues to communicate with the system using its familiar interface, completely unaware of the underlying JSON library.
- **Target Interface (IXMLAnalyticsHook / StockDataHandler)**: The standard interface the current application relies on to process XML data. It defines the `fetchAnalytics(xmlData:`

`String): AnalyticsResult` method.

- **Adaptee** (`JSONAnalyticsLibrary`): The smart 3rd-party analytics library we want to integrate. The challenge is that it only provides the `processJSONData(jsonData: String): AnalyticsResult` method, which is strictly incompatible with our current XML format.
- **Adapter** (`XMLToJsonAnalyticsAdapter`): The core component of this solution.
- The Adapter *implements* the `IXMLAnalyticsHook` interface, allowing the Client, `StockAppUI`, to interact with it seamlessly as if it were the original handler.
- The Adapter *holds a reference* to the `JSONAnalyticsLibrary`, through the `-jsonLib` private attribute, to utilize its analytics capabilities.

5.3 Data Flow

When a UI component needs to analyze data, the following execution flow occurs inside the Adapter, as illustrated in the diagram's note:

1. The Client passes the input data in XML format by calling the `fetchAnalytics(xmlData)` method.
2. Inside the Adapter, an internal operation translates the format: `jsonData = convertXmlToJson(xmlData)`.
3. Once the valid JSON data is generated, the Adapter delegates the actual processing to the 3rd-party analytics library by calling `return jsonLib.processJSONData(jsonData)`.
4. The final `AnalyticsResult` is returned to the Adapter, which then seamlessly passes it back to the calling UI component.

6 Implementation

The following C++ implementation demonstrates how the Adapter Pattern solves the stock analytics problem. The existing UI depends only on the XML-based `IXMLAnalyticsHook` interface, while the adapter internally converts XML data into JSON and delegates processing to the third-party `JSONAnalyticsLibrary`.

```
1 #include <iostream>
2 #include <string>
3 #include <memory>
4
5 // -----
6 // 1. Common Data Structure
7 // -----
8 // A mock structure representing the returned analytics data
9 struct AnalyticsResult {
10     std::string resultData;
11 };
12
13 // -----
14 // 2. Adaptee (The 3rd-Party Library)
15 // -----
16 // The smart analytics library that ONLY accepts JSON.
17 class JSONAnalyticsLibrary {
18 public:
19     AnalyticsResult processJSONData(const std::string& jsonData) {
20         std::cout << "[JSONAnalyticsLibrary] Processing JSON: " << jsonData << "\n";
21         // Mock processing logic
22         return AnalyticsResult{"Advanced JSON Analytics Generated"};
23     }
24 };
25
26 // -----
27 // 3. Target Interface
28 // -----
29 // The interface our current app expects (works with XML).
30 class IXMLAnalyticsHook {
31 public:
32     virtual ~IXMLAnalyticsHook() = default;
33     virtual AnalyticsResult fetchAnalytics(const std::string& xmlData) = 0;
34 };
35
36 // -----
```

```
37 // 4. Adapter
38 // -----
39 // Translates XML requests to JSON requests.
40 class XMLToJsonAnalyticsAdapter : public IXMLAnalyticsHook {
41 private:
42     std::shared_ptr<JSONAnalyticsLibrary> jsonLib;
43
44     // Internal mock method for XML to JSON conversion
45     std::string convertXmlToJson(const std::string& xmlData) {
46         std::cout << "[Adapter] Converting XML data to JSON format...\n";
47         return "{ \"converted_from_xml\": true }";
48     }
49
50 public:
51     // Dependency Injection via constructor
52     XMLToJsonAnalyticsAdapter(std::shared_ptr<JSONAnalyticsLibrary> library)
53         : jsonLib(library) {}
54
55     // Implementing the target interface
56     AnalyticsResult fetchAnalytics(const std::string& xmlData) override {
57         std::cout << "[Adapter] Received XML: " << xmlData << "\n";
58
59         // Step 1: Convert XML to JSON
60         std::string jsonData = convertXmlToJson(xmlData);
61
62         // Step 2: Pass to the JSON library and return the result
63         return jsonLib->processJSONData(jsonData);
64     }
65 };
66
67 // -----
68 // 5. Client
69 // -----
70 // The UI component that relies on the Target Interface.
71 class StockAppUI {
72 private:
73     std::shared_ptr<IXMLAnalyticsHook> analyticsHook;
```

```
74
75 public:
76     StockAppUI(std::shared_ptr<IXMLAnalyticsHook> hook) : analyticsHook(hook) {}
77
78     void renderDashboard() {
79         std::cout << "\n--- Rendering Dashboard ---\n";
80         std::string mockXML = "<stock><symbol>AAPL</symbol><price>150</price></stock>";
81
82         // The UI interacts blindly with the interface, unaware of the JSON library
83         AnalyticsResult result = analyticsHook->fetchAnalytics(mockXML);
84
85         std::cout << "[StockAppUI] Received result: " << result.resultData << "\n";
86     }
87
88     void displayCharts() {
89         std::cout << "[StockAppUI] Drawing beautiful charts using the fetched analytics...\n";
90     }
91 };
92
93 // -----
94 // 6. Main Execution
95 // -----
96 int main() {
97     // 1. Initialize the 3rd-party JSON library (Adaptee)
98     auto smartJSONLibrary = std::make_shared<JSONAnalyticsLibrary>();
99
100     // 2. Initialize the Adapter, passing the JSON library to it
101     auto adapter = std::make_shared<XMLToJsonAnalyticsAdapter>(smartJSONLibrary);
102
103     // 3. Initialize the Client (UI), passing the Adapter as the XML hook
104     StockAppUI app(adapter);
105
106     // 4. Execute Client logic
107     app.renderDashboard();
108     app.displayCharts();
109
110     return 0;
```

111

}

In this implementation, `StockAppUI` never calls `JSONAnalyticsLibrary` directly. It only depends on `IXMLAnalyticsHook`, while `XMLToJsonAnalyticsAdapter` performs the conversion and forwards the request to the JSON analytics library.

7 Quizzes

Question 1: What is the main purpose of the Adapter Design Pattern?

- A. To create many related objects without specifying their concrete classes.
- B. To convert one interface into another interface that the client expects.
- C. To ensure that a class has only one instance.
- D. To separate an algorithm from the object that uses it.

Answer: B

Explanation: The Adapter Pattern allows incompatible interfaces to work together by placing a translation layer between the client and the adaptee.

Question 2: In the stock analytics example, which class plays the role of the **Adaptee**?

- A. `StockAppUI`
- B. `IXMLAnalyticsHook`
- C. `JSONAnalyticsLibrary`
- D. `XMLToJsonAnalyticsAdapter`

Answer: C

Explanation: The adaptee is the existing or third-party component with a useful but incompatible interface. Here, `JSONAnalyticsLibrary` only accepts JSON data.

Question 3: In the implementation, why does `XMLToJsonAnalyticsAdapter` implement `IXMLAnalyticsHook`?

- A. So the client can keep calling the XML-based interface it already understands.
- B. So the third-party JSON library can directly render the dashboard.
- C. So XML data never needs to be converted.
- D. So the adapter can replace all UI components.

Answer: A

Explanation: The client depends on `IXMLAnalyticsHook`. By implementing this target interface, the adapter can be used by `StockAppUI` without changing the UI code.

Question 4: Which sequence best describes the data flow in the Adapter solution?

- A. Client sends JSON → adapter converts it to XML → XML hook processes it.
- B. Client sends XML → adapter converts it to JSON → JSON library processes it.
- C. JSON library sends XML → client converts it to JSON → adapter stores it.
- D. Adapter deletes XML → client rewrites itself → JSON library is ignored.

Answer: B

Explanation: The UI passes XML into `fetchAnalytics`. The adapter converts the XML into JSON and delegates the analytics work to `processJSONData`.

Question 5: Which problem of the naive solution does the Adapter Pattern most directly avoid?

- A. Rewriting many existing UI components to work directly with JSON.
- B. Creating an interface for the existing application.
- C. Using a third-party analytics library.
- D. Returning analytics results to the dashboard.

Answer: A

Explanation: Instead of changing every client that expects XML, the adapter hides the JSON-specific library behind the existing XML-based interface.

Question 6: What is the key difference between an **Object Adapter** and a **Class Adapter**?

- A. An Object Adapter uses composition, while a Class Adapter uses inheritance.
- B. An Object Adapter cannot translate data, while a Class Adapter can.
- C. An Object Adapter changes the client code, while a Class Adapter never does.
- D. An Object Adapter works only with XML, while a Class Adapter works only with JSON.

Answer: A

Explanation: The Object Adapter wraps an adaptee object and delegates calls to it. The Class Adapter inherits from the target and adaptee, which requires a language that supports the needed inheritance model.

Question 7: Which design principle is best supported by using the adapter instead of modifying the legacy XML-based application?

- A. Open-Closed Principle
- B. Single Instance Principle
- C. Last-In, First-Out Principle
- D. Principle of Least Recently Used

Answer: A

Explanation: The existing application can remain closed for modification while being extended with new JSON analytics behavior through the adapter.

Question 8: Which scenario is the best candidate for the Adapter Design Pattern?

- A. A program needs to sort an array of integers faster.

- B. A client expects XML input, but a useful third-party service only accepts JSON input.
- C. A class needs to restrict object creation to one instance.
- D. A factory needs to decide which product class to instantiate.

Answer: B

Explanation: The Adapter Pattern is appropriate when two components should collaborate but their interfaces or data formats are incompatible.

8 References

1. Refactoring.Guru, “Adapter,” *Design Patterns*. Available: <https://refactoring.guru/design-patterns/adapter>.
2. GeeksforGeeks, “Adapter Design Pattern.” Available: <https://www.geeksforgeeks.org/adapter-pattern/>.
3. TutorialsPoint, “Adapter Pattern,” *Design Patterns*. Available: https://www.tutorialspoint.com/design_pattern/adapter_pattern.htm.

9 Appendices